Week 5 Updates

Old architecture based on LorentzNet

x = Cartesian particle 4-momenta, $\phi_\cdot$ = learnable functions, $\psi(\cdot) = \mathrm{sgn}(\cdot)\log(|\cdot|+1)$,
c = hyperparameter, N = # particles, $||\cdot||, \langle\cdot\rangle$ = Minkowski norm and inner product

LGEB ([1])

$$m_{ij}^l = \phi_e(h_i^l, h_j^l, \psi(||x_i^l - x_j^l||), \psi(\langle x_i^l, x_j^l \rangle))$$
$$m_{ij}^l = \phi_m(m_{ij}^l) \cdot m_{ij}^l$$

$$x_i^{l+1} = x_i^l + \frac{c}{N} \sum_{j\in[N]} \phi_x(m_{ij}^l) \cdot |x_i - x_j|$$

$$h_i^{l+1} = h_i^l + \phi_h(h_i^l, \sum_{j\in[N]} \phi_w(m_{ij}^l) m_{ij}^l)$$

Simple LE block

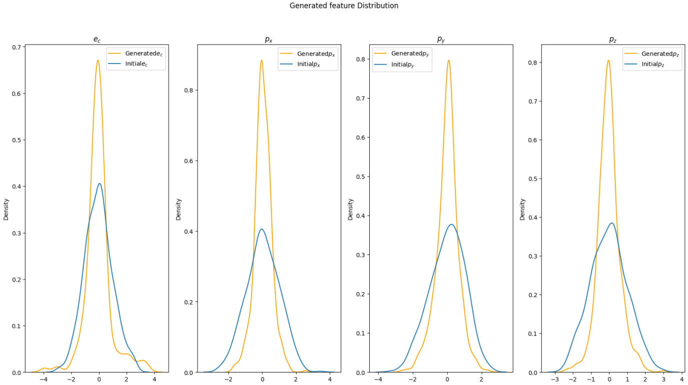
$$m_{ij}^l = \phi_e(\psi(||x_i^l - x_j^l||), \psi(\langle x_i^l, x_j^l \rangle))$$
$$m_{ij}^l = \phi_m(m_{ij}^l) \cdot m_{ij}^l$$

$$x_i^{l+1} = x_i^l + \frac{c}{N} \sum_{j\in[N]} \phi_x(m_{ij}, g) \cdot |x_j^l - x_i^l|$$

g = global feature vector (currently time embedding $\phi_t(t)$)

Performance

6 layers, 100 epochs, gluons only. 500 generated samples over 32 steps.



New architecture I

1. Embed scalar time value to random Fourier features through two fully connected layers with 32 and 64 nodes (following [2])
2. Concatenate time embedding with jet conditioning information, which consists of one-hot encoded type, η, mass, p_T, number of constituents
3. Pass this through a fully connected layer of size 64 to produce the initial global embeddings g^0
4. Concatenate g^0 with features $x_1^0 \ldots x_N^0$ initialized from the initial distribution p_0.
5. Now, we use L Lorentz-group equivariant layers.
   - 5.1 The input to the $l+1$-th layer is g^l and $x_1^l \ldots x_N^l$.
   - 5.2 Calculate messages $m_{ij}^l = \phi_e^l(\psi(||x_i^l - x_j^l||), \psi(\langle x_i^l, x_j^l \rangle))$
   - 5.3 Update the global embedding

$$g_{l+1} = \phi_g^l(g^0, g^l, \frac{c}{N} \sum_{i,j \in [N]} m_{ij}^l \cdot \phi_m^l(m_{ij}), \sum_{i,j \in [N]} m_{ij}^l \cdot \phi_m^l(m_{ij}))$$

New architecture II

5.4 Feed in the updated global embedding as well as the original embedding to update the graph embeddings

$$x_i^{l+1} = x_i^l + \sum_{j \in N} \phi_x^l(g^0, g^{l+1}, m_{ij}) \cdot x_j^l$$

5.5 Pass the results g^{l+1} and x_i^{l+1} to the next layer

6. After the final LE layer, run the particle representations through a final round of message-passing with the final global embedding

$$x_i^{L+1} = x_i^L + \sum_{j \in N} \phi_x^L(g^0, g^L, m_{ij}) \cdot x_j^L$$

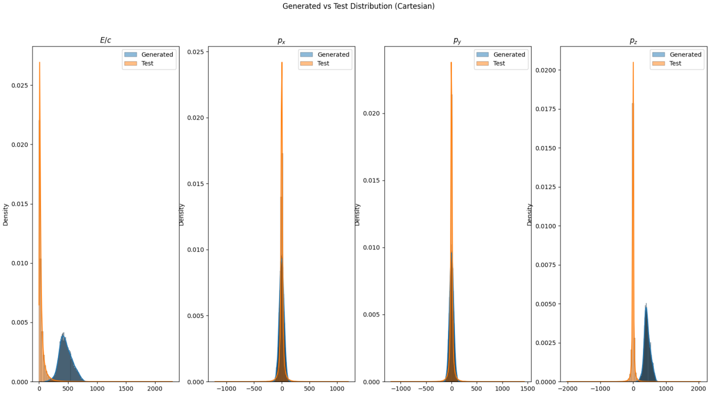
7. The final output $v_\theta(t, x^0)_i = x_i^L - x_i^0$

Jet conditioning implementation

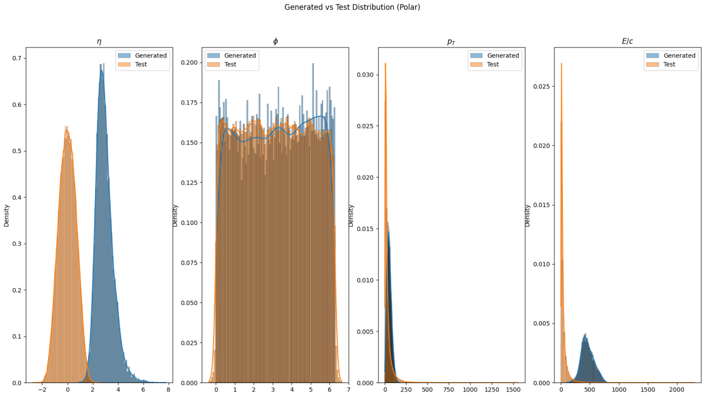
- During training, use the real features corresponding to each jet.
- During inference:
  - Use the normalizing flow to generate jet attributes
  - Round the number of jet constituents and clamp to [4, 150]
  - Generate masks corresponding to the desired number of constituents

Preliminary results I

100 epochs, 6 layers, gluons only. 500 generated samples with 32 steps.



Preliminary results II



Preliminary results III

Table: Metrics relative to test set

Metric	Value	MPGAN Value
Covariance (↑)	0.068	0.240
MMD (↓)	2.018	0.045

Inspiration for new architecture I

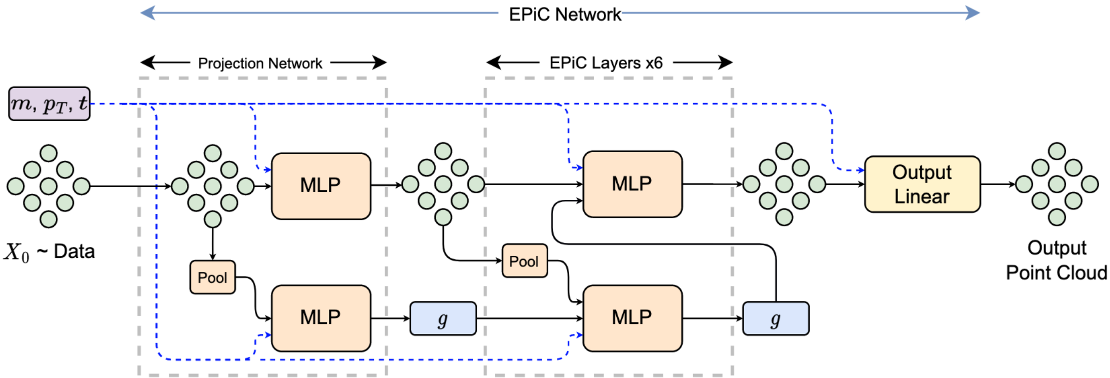
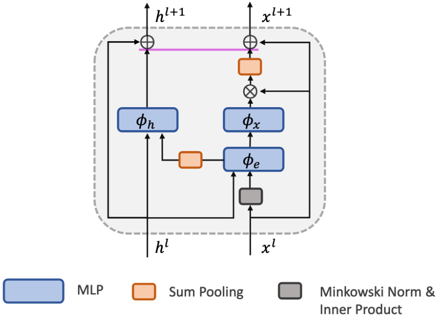
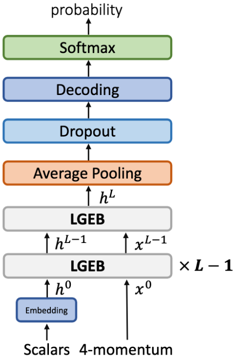


Figure: EPiC-FM [3]

Inspiration for new architecture II



Lorentz Group Equivariant Block (LGEB)



LorentzNet

Inspiration for new architecture III

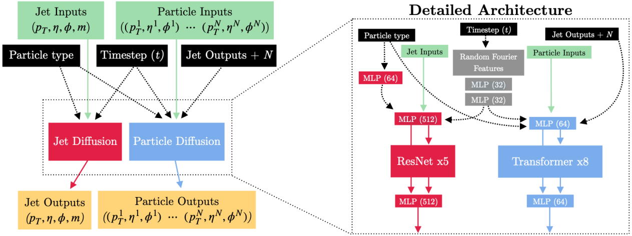


Figure: FPCD [2]

Implementation details for next training runs

- AdamW [4] optimizer
- Learning rate: linear warm-up followed by cosine annealing (same as [3])
- LeakyRELU activation (negative slope $= 0.01$)
- Zero padding for variable-sized particle clouds

Zero padding

In each layer:

- Calculate edge features for all particles and multiply by expanded mask
- Multiply results of $\phi_e(m_{ij})$, $\phi_x(m_{ij})$ by expanded mask
- Keep track of valid edges for average pooling
- To guarantee the mask is respected, multiply node updates and final nodes by mask before passing to next layer

Learning rate

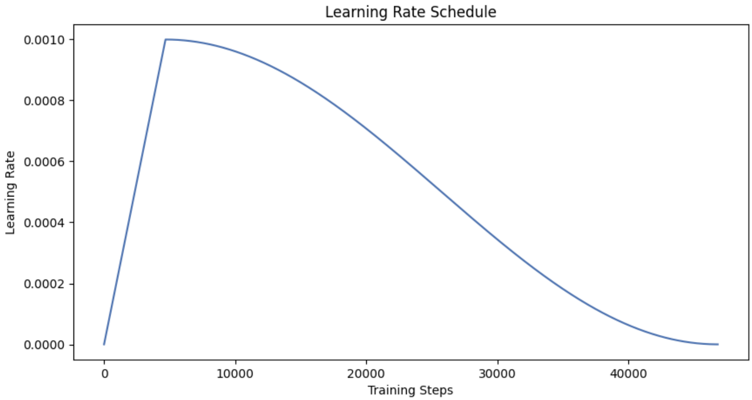


Figure: Linear warm-up followed by cosine annealing

This week I

- Run jobs today:
  - Entire dataset; $\approx$ 50 epochs; $\approx$ 8 layers
  - Entire dataeset; $\approx$ 1000 epochs; $\approx$ 12 layers
- Implement parallelized training with Hugging Face Accelerate
- Set up sampling pipeline while training the model to get an indication of how performance improves over time (since FM loss tends to be meaningless after the first epoch)
- Theoretical problems with equivariance
  - Currently initializing from normal distribution
  - Go over E(3)-equivariant papers in detail and try to develop something similar to CoM trick
- Set up plots typically used in jet generation papers
- Mean flow setup (should be the first paper to use it for jets)

References I

[1] Shiqi Gong et al. "An Efficient Lorentz Equivariant Graph Neural Network for Jet Tagging". In: *Journal of High Energy Physics* 2022.7 (July 2022), p. 30. ISSN: 1029-8479. DOI: 10.1007/JHEP07(2022)030. (Visited on 06/30/2025).

[2] Lemeng Wu et al. "Fast Point Cloud Generation with Straight Flows". In: *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Vancouver, BC, Canada: IEEE, June 2023, pp. 9445–9454. ISBN: 979-8-3503-0129-8. DOI: 10.1109/CVPR52729.2023.00911. (Visited on 06/23/2025).

[3] Erik Buhmann et al. *EPiC-ly Fast Particle Cloud Generation with Flow-Matching and Diffusion*. Sept. 2023. DOI: 10.48550/arXiv.2310.00049. arXiv: 2310.00049 [hep-ph]. (Visited on 03/13/2025).

[4] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. Jan. 2019. DOI: 10.48550/arXiv.1711.05101. arXiv: 1711.05101 [cs]. (Visited on 07/22/2025).